

Atividade – SOLID

1- A classe **Pedido** original atua como uma "Classe Deus" (*God Class*), acumulando responsabilidades que deveriam estar distribuídas. Ela possui 6 responsabilidades distintas:

- **Gerenciamento do pedido e cálculo do total bruto:** Armazena os dados básicos e processa a soma dos itens . *Classe sugerida:* **Pedido**.
- **Cálculo de regras de desconto:** Contém a lógica de precificação promocional . *Classe sugerida:* **CalculadoraDesconto**.
- **Persistência de dados:** Realiza operações diretas de banco de dados . *Classe sugerida:* **PedidoRepository**.
- **Notificação ao cliente:** Centraliza a comunicação e envio de mensagens . *Classe sugerida:* **NotificacaoService**.
- **Geração de relatório:** Formata os dados do pedido em formato de texto estruturado . *Classe sugerida:* **RelatorioPedido**.
- **Impressão e exportação:** Trata do direcionamento do relatório para saídas físicas ou digitais . *Classe sugerida:* **ImpressaoService**.

2- A classe possui 6 razões para mudar, correspondentes a cada uma de suas responsabilidades:

1. Alteração na estrutura do pedido ou na forma de calcular o total bruto dos itens .
2. Inclusão, remoção ou alteração de regras e cupons de desconto .
3. Mudança na tecnologia ou na estrutura do banco de dados (ex: migrar de MySQL para outro banco) .
4. Alteração no canal de envio (ex: SMS) ou no texto da mensagem enviada ao cliente .
5. Modificação no layout ou nas informações exibidas no relatório de texto .
6. Adição ou modificação nos formatos de saída do documento (ex: novos drivers de impressora) .

3- **Pedido, CalculadoraDesconto, PedidoRepository, NotificacaoService, RelatorioPedido, ImpressaoService**

4-

```
class EstrategiaDesconto {
    aplicar(total) {
        return total;
    }
}

class DescontoVip extends EstrategiaDesconto {
    aplicar(total) {
        return total * 0.8;
    }
}

class DescontoCupom10 extends EstrategiaDesconto {
    aplicar(total) {
        return total * 0.9;
    }
}

class SemDesconto extends EstrategiaDesconto {
    aplicar(total) {
        return total;
    }
}

class CalculadoraDesconto {
    calcular(total, estrategiaDesconto) {
        return estrategiaDesconto.aplicar(total);
    }
}
```

5-

- **Código refatorado:** Nenhuma linha do código existente precisa ser alterada. Basta criar uma nova classe isolada (ex: DescontoAniversario) que estende EstrategiaDesconto.
- **Código original:** Seria necessário modificar diretamente a classe Pedido, localizando o método aplicarDesconto e adicionando uma nova instrução condicional if . Isso violaria o OCP.

6- O problema deve ser resolvido através de **composição**. O cálculo de desconto deve ser uma dependência injetada no fluxo do pedido, permitindo que diferentes estratégias sejam associadas dinamicamente sem engessar a classe por meio de herança.

7- Para manter a consistência do sistema, o método deveria retornar 0

8- Se o método lançar um erro, haverá violação do LSP . O princípio determina que subclasses devem poder substituir suas classes base sem quebrar o comportamento esperado. Se o sistema iterar sobre uma lista genérica de pedidos e um deles disparar uma exceção inesperada em um método padrão, a execução será interrompida.

9-A solução consiste em remover a responsabilidade de frete da classe base comum. O frete deve ser tratado apenas onde faz sentido:

- A classe base Pedido conterá apenas os atributos universais.
- Cria-se uma interface ou classe especializada chamada PedidoFisico que implementa a lógica de entrega e frete, enquanto PedidoDigital não herda esses comportamentos.

10- Não. Um serviço especializado em gerar arquivos PDF não deve ser obrigado a carregar dependências ou implementar métodos relacionados a impressoras físicas . Interfaces robustas demais criam acoplamentos desnecessários, violando o ISP

11-

```
class Imprimivel {
    imprimir(conteudo) {
        throw new Error("Método imprimir não implementado");
    }
}

class Exportavel {
    exportar(conteudo) {
        throw new Error("Método exportar não implementado");
    }
}
```

12-

```
class Pedido {
  constructor(id, cliente, itens, repository, notificationService) {
    this.id = id;
    this.cliente = cliente;
    this.itens = itens;
    this.status = 'pendente';
    this.repository = repository;
    this.notificationService = notificationService;
  }

  calcularTotal() {
    return this.itens.reduce((acc, item) => acc + (item.preco * item.quantidade), 0);
  }

  salvar() {
    this.repository.salvar(this);
  }

  notificarCliente() {
    this.notificationService.enviar(this.cliente.email, `Pedido ${this.id} confirmado!`);
  }
}
```

13-

```
repositoryMock = {
  salvar: false,
  chamouSalvar: false
};

notificationMock = { enviar() {} };

const pedido = new Pedido(1, { email: 'teste@email.com' }, [], repositoryMock, notificationMock);
pedido.salvar();

expect(repositoryMock.chamouSalvar).toBe(true, "Erro: O repositório deveria ter sido acionado.");
```

14-

Antes da refatoração: Era necessário modificar o código interno da classe Pedido para trocar a classe instanciada internamente .

Após a refatoração: Nenhuma classe existente precisa ser modificada. Basta criar a nova classe SMSService seguindo a mesma interface e passá-la como parâmetro no momento de instanciar o Pedido.